# Fundamentos de Programação

Python

1º Semestre 1º Ano

## Computador

→ O computador é uma máquina cuja função é manipular símbolos: são automáticos, universais, eletrónicos e digitais;

* Uma vez alimentado c/a informação necessária, trabalha por si só, sem a intervenção humana
  - Recebe um programa → conjunto de instruções quanto ao modo de resolver o problema
    - escritas numa notação compreendida pelo computador
    - Especificam exatamente como o trabalho deve ser executado

* Pode efetuar qualquer tarefa cuja solução possa ser expressa através de um programa

## Operações Elementares

→ De acordo com a tese de Church-Turing, qualquer computação pode ser baseada num pequeno número de operações elementares:

1. Operações de entrada de dados;
2. Operações de saída de dados;
3. Operações matemáticas;
4. Execução condicional;
5. Repetição

→ A programação corresponde a dividir um programa grande e complexo em vários problemas, até poderem ser expressos por operações elementares.

* Efetua operações sobre informação que é codificada recorrendo a duas grandezas discretas (0 e 1) e não grandezas que variam de modo contínuo.

## Algoritmos

→ Um programa é caracterizado matematicamente como um algoritmo, escrito numa linguagem que é entendida pelo computador

→ É uma sequência finita de instruções bem definidas e não ambíguas, cada uma das quais pode ser executada mecanicamente num período de tempo finito com uma quantidade de esforço finita.

* Existe uma ordem pela qual as instruções aparecem no algoritmo, e que estas instruções são em número finito;
* Significado de cada uma das instruções é claro, não existem múltiplas interpretações;
* A execução das instruções não requer imaginação por parte dos executantes;
* A execução de cada uma das instruções termina.

→ Um algoritmo está sempre associado à Resolução de um dado problema; a execução das instruções garante que o seu objetivo é atingido.

## Características de um Algoritmo

→ Sequências finitas de instruções que devem ser seguidas de modo a atingir um determinado objetivo;

→ Um algoritmo deve possuir três características: ser rigoroso, ser eficaz e ter a garantia de terminar:

1. Deve especificar exacta e rigorosamente o que deve ser feito, não havendo lugar para ambiguidade; Para evitar a ambiguidade inerente à linguagem natural criam-se novas linguagens para exprimir os algoritmos de um modo rigoroso.
2. Cada instrução do algoritmo deve ser suficientemente básica e bem compreendida de modo a poder ser executada num intervalo de tempo finito, com uma quantidade de esforço finita;
3. O algoritmo deve levar a uma situação em que o objetivo tenha sido atingido e não existam mais instruções para serem executadas;

## Programas e Algoritmos

→ Um algoritmo escrito de modo a poder ser executado por um computador tem o nome de programa

→ tipicamente, o computador começa por ler valores para algumas variáveis, após o que efetua operações sobre essas variáveis, possivelmente atribuindo valores a novas variáveis e, finalmente, chega a um conjunto de valores que constituem o resultado programa;

→ As ações devem ser suficientemente elementares para poderem ser executadas facilmente pelo agente que executa o algoritmo; Estas ações elementares podem ser divididas isto é, podem referir-se a um conjunto de ações mais simples a serem executadas numa sequência bem definida.

## Linguagens de Programação

→ Linguagem usada para escrever programas de computador;

→ Podem ser classificadas em linguagens máquina, "assembly" e linguagens de alto nível.

→ A linguagem máquina é a linguagem usada para comandar diretamente as ações do computador; são constituídas por uma sequência de símbolos discretos, correspondentes à existência ou à ausência de sinal e manipulam diretamente entidades dentro do computador ⟶ é difícil de usar e compreender por humanos e varia de computador para computador;

→ A linguagem "assembly" é semelhante à linguagem máquina, diferindo desta no sentido em que usa nomes simbólicos com significado para humanos;

→ As linguagens de alto nível aproximam-se das linguagens que os humanos usam e podem ser utilizadas em computadores diferentes.

→ Para que os computadores possam "entender" os programas escritos numa linguagem de alto nível, existem programas que traduzem as instruções de linguagens de alto nível em linguagem máquina, chamados processadores da linguagem → compilação
→ Interpretação (Python)

## Sintaxe e Semântica

→ A sintaxe é o conjunto de regras que definem as relações válidas entre os componentes da linguagem → nada diz em relação ao significado das frases da linguagem // Como apenas se preocupa com o processo de combinação dos símbolos de uma linguagem pode ser facilmente formalizada.

- Notação BNF:
  - Para designar um componente da linguagem escrevemo-lo entre parênteses: "<" e ">"; Os nomes entre <e> são designados por símbolos não terminais
  - O símbolo "|" (ou) representa possíveis alternativas
  - O símbolo "::=" serve para definir componentes da linguagem
  - O símbolo + imediatamente a seguir a um símbolo não terminal significa que esse símbolo pode ser repetido uma ou mais vezes.
  - " * " significa que esse símbolo pode ser repetido ZERO ou mais vezes;
  - { } significa que os símbolos entre elas são opcionais

- exemplo:

  <número binário> ::= <dígito binário> | <dígito binário> <número binário>

  <dígito binário> ::= 0 | 1

  ↳ PODE ORIGINAR RESULTADOS COMO:

  0 01 10110 etc...
  1 011 01111011

- Nas linguagens de programação, a ambiguidade sintética não pode existir; A notação utilizada para definir formalmente uma linguagem, no caso da notação BNF, < >, | , ::=, {}, +, *, os símbolos não terminais e os símbolos terminais, é denominada metalinguagem, visto ser a linguagem que utilizamos para falar acerca de outra linguagem;

→ A semântica de uma linguagem define qual o significado de cada frase da linguagem. Cada frase em Python tem uma semântica, a qual corresponde às ações tomadas pelo Python ao executar essa frase. Esta semântica é definida por regras para extrair o significado de cada frase.

## Tipos de Erros num Programa

→ Podem ser de natureza sintática ou semântica;

★ Sintáticos - resultam do facto de o programador não ter escrito as frases do seu programa de acordo com as regras da gramática da linguagem de programação utilizada; A deteção é feita pelo processador da linguagem, o qual fornece normalmente um diagnóstico sobre o que provavelmente está errado;

★ Semânticos - são erros, em geral, muito mais difíceis de detetar; Resultam do facto do programador não ter expressado corretamente, através da linguagem de programação, as ações a serem executadas; podem manifestar-se pela geração de uma mensagem de erro

FPZ

[illegible] execução de um programa, [illegible] ciclos que nunca terminam;

★ Ao processo de deteção e correção, dá-se o nome de depuração → "debugging" / e aos erros chama-se "bugs" || Para desenvolver programas são necessárias duas competências fundamentais, a capacidade de Resolução de problemas e a capacidade de depuração

## Expressões

→ É uma entidade computacional que tem um valor;

→ Pode ser uma constante, uma expressão composta, um nome ou uma aplicação de função em BNF, uma expressão é definida do seguinte modo:

<expressão> ::= <constante> | <expressão composta> | <nome> | <aplicação de função> |

## Constantes

→ Números, valores lógicos ou cadeias de caracteres;

→ O valor de uma constante é a própria constante;

→ A Representação externa de uma entidade corresponde ao modo como nós visualizamos essa entidade, independentemente do modo como esta é representada internamente no computador;

R. externa → R. interna

1958 → 11110100110

★ Tipos de constantes:

1. Números inteiros (nos seu parte decimal)
2. Números reais (nos com parte decimal
3. Valores Lógicos (True e False)
4. Cadeias de caracteres (sequência de caracteres), aparecem delimitadas por aplicas "; o seu comprimento é o número de caracteres do seu conteúdo.

## Expressões Compostas

→ Existem operações embutidas / pré-definidas / primitivas;

→ As operações embutidas podem ser utilizadas através do conceito de expressão composta; corresponde ao conceito de aplicação de uma operação a operandos; é constituída por um operador e um certo número de operandos; os operadores podem ser unários (se têm apenas 1 operando) ou binários (se têm dois operandos ex: + *).

→ Ordem de aplicação de operadores numa expressão:

1. Lista de prioridades dos operadores;
2. Ordem de aplic. dos op. quando se encontram dois operadores c/a mesma prioridade → Da esquerda p/ a direita

Prioridade

máxima ↑ mínima

funções
not, - (simétrico)
*, /, //, %.
+, - (subtração)
<, >, ==, >=, <=, != 
and
OR

## Tipos Elementares de Informação

→ Um tipo de informação é caracterizado por um conjunto de entidades (valores) e um conjunto de operações aplicáveis a essas entidades. Ao conjunto de entidades dá-se o nome de domínio do tipo. Cada uma das entidades do domínio do tipo é designada por elemento do tipo.

→ Dois Grupos: → Inteiro, Real e Lógico

1. Elementares - as suas constantes (elementos) serem tomadas como indecomponíveis, por exemplo, "Verdadeiro" e "falso" (tipo lógico);
2. Estruturados - as suas constantes são constituídas por um agregado de valores;

## O Tipo Inteiro

→ Os números inteiros, designados por int, são números sem parte decimal, podem ser positivos, negativos ou ZERO;

| Operação | Valor |
|---|---|
| $e_2 + e_2$ | soma |
| $e_1 - e_2$ | subtração |
| $-e$ | simétrico |
| $e_1 * e_2$ | multiplicação |
| $e_1 // e_2$ | divisão inteira |
| $e_1 \% e_2$ | Resto da divisão inteira |
| abs(e) | valor absoluto |

FP

## O Tipo Real

→ Designam-se por float, são números com parte decimal; existe a notação decimal e a notação científica;

| Operação | Valor |
|---|---|
| e2 + e2 | soma |
| e1 - e2 | subtração |
| -e | simétrico |
| e1 * e2 | produto |
| e1 / e2 | divisão |
| abs(e) | valor absoluto |

| Operação | Tipo Arg | Tipo valor | Operação |
|---|---|---|---|
| Round(e) | Real | int | Inteiro mais próximo do Real e |
| int(e) | Real | int | Parte inteira do Real e |
| float(e) | int | Real | Número real correspondente a e |

## Tipo Lógico

→ Designado por bool, pode assumir dois valores: True ou False;

→ As operações podem ser:

**unárias** (ou) **binárias**

~ Produzem um valor lógico a partir de um valor lógico. Apenas existe o not, que muda o valor lógico;

~ Aceitam dois argumentos do tipo lógico e produzem um valor do tipo lógico; Existe a conjunção e a disjunção;

↳ and
os dois são True → True

↳ or
os dois são False → False

## Nomes e Atribuição

→ A associação entre um nome e um valor é Realizada através da instrução de atribuição, que recorre à operação embutida =, o operador de atribuição;

| Operação | Valor |
|---|---|
| e1 == e2 | igual |
| e1 != e2 | diferente |
| e1 > e2 | maior |
| e1 >= e2 | maior ou igual |
| e1 < e2 | menor |
| e1 <= e2 | menor ou igual |

(Valor: VALOR TRUE ou FALSE)

## Predicados e Condições

→ Predicado: operação que produz resultados do tipo lógico;

→ Condição: expressão cujo valor é do tipo lógico;

## Leitura de Dados

→ input ('informação')

```
>>> input ('->')
-> 5
'5'
```

* Inserido através do teclado

| caráter Escape | significado |
|---|---|
| \\ | Barra ao contrário (\) |
| \' | Plica (') |
| \" | Aspas |
| \a | Toque de Campainha |
| \b | Retrocesso de um espaço |
| \f | Salto de página |
| \n | Salto de linha |
| \r | "Return" |
| \t | Tabulação horizontal |
| \v | Tabulação Vertical |

→ eval(characteres) → devolve um número / avalia

```
>>> eval('5')
5
```

## Escrita de Dados

→ print(info)

=>

```
>>> a = 10
>>> b = 15
>>> print('a=', a, 'b =', b)
a= 10 b= 15
```

```
>>> print('a=', a, '\nb =', b)
a= 10
b= 15
```

ff

# Programas, Instruções e Sequenciação

→ Ao nível de linguagem máquina, existem 2 tipos de estruturas de controle: a sequenciação e o salto;

* As instruções são executadas pela ordem em que aparecem no programa;
* Transferência da execução para qualquer ponto do programa;

→ Em linguagens de alto nível para além destas estruturas existem outras, como a seleção e a Repetição.

BNF

<instruções> ::= <instrução> CR |
<instrução> CR <instruções>

CR → corresponde ao fim da linha

Python

```
nota_1 = 17
nota_2 = 18
media = (nota_1 + nota_2) /2
print ('A média é', media)
```
↳ A média é 17,5

## Seleção

→ Execução condicional de instruções

→ if → permite a seleção entre duas ou mais alternativas

```
>>> if nota > 5
        print ('Bom Trabalho')
```

! atenção à tabelação !

→ else → se não cumprir o if

```
>>> nota = 15
>>> if nota > 16
        print ('Bom trabalho')
    else:
        print ('Podes fazer melhor')
Podes fazer melhor
```

→ elif → else com uma condição

## Repetição

→ Repetição da execução de um grupo de instruções ou de todo o programa;

→ Ciclo finito

→ While → enquanto uma expressão do tipo lógico for verdadeira

→ Instrução BREAK → quebra o ciclo

```
soma = 0
num = eval(input('Escreva um inteiro\n (-1 para terminar)'))

while num != -1:
    soma = soma + num
    num = eval(input('Escreva um inteiro\n (-1 p/ terminar)'))
print ('A soma é:', soma)
```

→ A quantidade de vezes que é preciso repetir o ciclo é desconhecida;

## Funções

→ Definir as funções por compreensão: chegar ao resultado final através dos valores de entrada das suas variáveis e a indicação de um processo de cálculo (f(x) = x * x);

1. A definição de uma função é feita fornecendo um nome para a função, uma indicação do seu domínio (f(5)).
2. A aplicação da função é feita fornecendo o nome da função e um elemento do

```
def quadrado (x):          ← parâmetro formal
    Return x * x           → corpo da função
```

```
>>> quadrado (7)
49
```

1. Avaliação dos parâmetros
2. Associa os valores inseridos com os parâmetros definidos
3. Executa as instruções

CRIA UM AMBIENTE LOCAL

→ Return → apenas dentro de funções / termina a execução do corpo da função;

## Abstração Procedimental

→ Consiste em dar um nome à sequência de ações que serve para atingir um objetivo, e utilizando esse nome cada vez que desejamos atingir esse objetivo sem ter de especificar de novo todas as ações.

→ Permite desenvolver programas complexos, sem nos perdermos nos pormenores

## Erros

| Nome | Situação |
|---|---|
| AttributeError | Atributo ñ existente num objeto |
| ImportError | Biblioteca ñ existente |
| IndexError | Índice fora da gama de tuplo/lista |
| KeyError | Chave inexistente |
| SyntaxError | Eval / Input → enc. exp. sintaxe incorreta |

| Nome | Situação |
|---|---|
| NameError | Nome que não existe |
| ValueError | Argumento incorreto |
| ZeroDivisionError | Divisão p/ zero |

Raise: força a geração de um erro

FP!

## Estruturas de Blocos

→ Estruturas de blocos: Definir funções dentro das quais existem outras funções; um [illegible] pode ser considerada um bloco;

## Modules

→ coleção de funções agrupadas num único ficheiro

```
>>> import math → módulo
>>> math.pi → função
>>> math.sqrt(4)
   2.0
>>> math.sin(math.pi/2)
    1.0
```

| Python | Significado |
|---|---|
| pi | π |
| e | e |
| sin(x) | sin(x) |
| cos(x) | cos(x) |
| tan(x) | tan(x) |
| log(x) | ln(x) |
| exp(x) | $e^x$ |
| pow(x,y) | $x^y$ |
| sqrt(x) | $\sqrt{x}$ |
| ceil(x) | Maior inteiro >= a x |
| floor(x) | Maior inteiro <= a x |

## Tuplos

→ Sequência de elementos

→ Referimo-nos aos elementos através da posição que ocupam: índices → -1 é o último elemento; começam no 0

→ Representação: (), (1,2,3), (1, True), (1, ), (1, (1,1), 1)

```
>>> notas = (15, 6, 10, 12, 12)
>>> notas
(15, 6, 10, 12, 12)
>>> notas[0]
15
>>> notas[-2]
12
>>> i=1
>>> notas[i+1]
10
```

```
>>> a = ((1,2,3), 4, (5,6))
>>> a[0]
(1,2,3)
>>> a[0][1]
2
```

SÃO IMUTÁVEIS

| Operação | Valor |
|---|---|
| t1 + t2 | concatenação |
| t * i | Repetição i vezes do tuplo |
| t[i1 : i2] | sub-tuplo de t entre i1 e i2 - 1 |
| (*1) e in t | True se e pertence a t |
| e not in t | True se e não pertence a t |
| tuple(a) | Transforma o seu arg num tuplo |
| len(t) | Número de elementos do tuplo |

→ isinstance: tem o valor True apenas se o tipo da expressão do 1º arg corresponde ao seu 2º arg

```
>>> isinstance(3, int)
    True
```

## Ciclos Contados

→ for

```
>>> for i in (1,3,5):
        print(i)
1
3
5
```

É PREFERÍVEL AO WHILE

→ range → geração de sequência de elementos

1. range(e2) devolve a sequência contendo os inteiros entre 0 e e2 - 1 / ex: range(2) = (1,2)
2. range(e1, e2) devolve a sequência entre e1 e e2 - 1; ex: range(-3,3) = (-3, -2, -1, 0, 1, 2)
3. range(e1, e2, e3) devolve sequência começa em e1, nunca superior a e2 - 1, em que cada elemento é obtido pela soma do elemento anterior com e3 / ex: range(2, 20, 3) = (2, 5, 8, 11, 14, 17)

## Cadeias de Caracteres Revisitadas

→ STR IMUTÁVEIS

→ help(<nome>) mostra a definição associada a <nome> ~ delimitada por """ def """

(*1) Mesmas operações

→ ord(a) → recebe char e devolve o código decimal que o representa } unicode

→ chr(a) → recebe um nº positivo e devolve o caracter

```
>>> ord('R')
82
```

```
>>> chr(125)
'}'
```

## Listas

→ list: é uma sequência de elementos

| Operação | valor |
|---|---|
| l1 + l2 | concatenação |
| l * i | Repetição i x da lista |
| l[i1 : i2] | sub-lista entre i1 e i2-1 |
| del(els) | Remove os elementos especificados |

| Operação | valor |
|---|---|
| e in l | True se e pertence à lista |
| e not in l | True se e não pertence à lista |
| list(a) | Transforma o arg em lista |
| len(l) | Devolve o nº de elementos |

são mutáveis, logo lst = [1,2,3,4]

```
>>> lst[2] = 'a'
print lst
[1, 2, 'a', 4]
```

## Métodos de Passagem de Parâmetros

→ **Passagem por valor**: A função recebe o valor de cada um dos parâmetros e nenhuma informação adicional; exemplo:

```
def troca(x,y):
    x,y = y,x  # os valores são trocados
>>> x = 3
>>> y = 10      } criação de x e y no ambiente global
>>> troca(x,y) } local
>>> x
3
>>> y
10
```

UNIDIRECIONAL

→ Quando a função troca é executada, o ambiente local desaparece, voltando-se aos valores de x e y iniciais

→ **Passagem por Referência**: os parâmetros formais e concretos partilham o mesmo ambiente:

```
def troca_2(lst, i1, i2):
    lst[i1], lst[i2] = lst[i2], lst[i1]
>>> lista = [0,1,2,3,4,5]
>>> troca_2(lista, 2, 5)
>>> lista
[0,1,5,3,4,2]
```

## Algoritmos de Procura

→ **Procura sequencial**: Começar no primeiro elemento da lista e comparar sucessivamente o elemento procurado com o elemento na lista:

```
def procura(lst, chave):
    for i in range(len(lst)):
        if lst[i] == chave
            return i
    return -1
```

→ **Procura Binária**: Lista ordenada → considera-se o elemento do meio da lista x, se x > procura, está na 1ª metade → Repetir o processo p/ a 1ª metade.

```
def procura(lst, chave):
    linf = 0
    lsup = len(lst)-1
    while linf <= lsup:
        meio = (linf + lsup) // 2
        if chave == lst[meio]:
            return meio
        elif chave > lst[meio]:
            linf = meio + 1
        else:
            lsup = meio - 1
    return -1
```

## Algoritmos de Ordenação

→ Ordenação **interna** e **externa**

- **interna**: Ordenam um conjunto de elementos que estão simultaneamente armazenados em memória (ex: lista)
- **externa**: Ordenam elementos que devido à sua quantidade não podem estar simultaneamente em memória (ex: ficheiros)

→ **Ordenação por borbulhamento**: Percorrer os elementos a ordenar, comparando elementos adjacentes, trocando pares de elementos que se encontram fora de ordem;

→ **Ordenação Shell**: comparar e trocar elementos separados por um certo intervalo; esse intervalo é dividido ao meio e o processo repete-se com o novo intervalo

→ **Ordenação por Seleção**: em cada passagem colocar um elemento na sua posição correta;

## Funções Recursivas

→ Função definida em termos de si própria, exemplo:

Não recursiva:

```
def factorial(n):
    fact = 1
    for i in range(n, 0, -1):
        fact = fact * i
    return fact
```

Recursiva:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

→ Encadeamento:

```
f(3)
3 * f(2)
3*(2* f(1))
3*(2*(1* f(0)))
3*(2*(1*1))
3*(2*1)
3*2
6
```

1. Caso terminal: versão mais simples do problema para a qual a solução é conhecida;
2. Caso geral: o problema é definido em termos de uma versão mais simples de si próprio;

## Funções de Ordem Superior

→ funções como Parâmetros:

```
def somatorio (calc-termo, linf, prox, lsup):
    soma = 0
    while linf <= lsup:
        soma = soma + calc-termo (linf)
        linf = prox (linf)
    Return soma
```

```
def inc1 (x):
    Return x+1
def identidade (x):
    Return x
def inc2 (x):
    Return x+2
def inv-quadrado (x):
    Return 1 / quadrado (x)
```

```
>>> somatorio (identidade, 1, inc1, 10)
```

↳ Utilizam-se funções como parâmetros

lambda x: x+1, devolve o valor do seu argumento mais um

```
>>> somatorio (lambda x: x, 1, lambda x: x+1, 10)
```

lambda x: 1 / quadrado (x) = inv-quadrado (x)

```
def factorial (n):
    if n==0:
        Return 1
    else:
        Return n * factorial (n-1)
```

## Funcionais sobre Listas

→ Um transformador é um funcional que recebe como argumentos uma lista e uma operação aplicável aos alimentos da lista, e devolve uma lista em que cada elemento resulta da aplicação da operação ao elemento correspondente da lista original:

```
def transforma (tr, lista):
    Res = list ()
    for e in lista:
        Res = Res + [tr (e)]
    Return Res
```

→ Um filtro é um funcional que recebe como elementos uma lista e um predicado aplicável aos elementos da lista, e devolve a lista constituída apenas pelos elementos que satisfazem o predicado:

```
def filtra (teste, lista):
    Res = list ()
    for e in lista:
        if teste (e):
            Res = Res + [e]
    Return Res
```

→ Um acumulador recebe como argumentos uma lista e uma operação aplicável aos elementos da lista, e aplica sucessivamente essa operação aos elementos da lista, devolvendo o resultado da aplicação da operação a todos os elementos da lista:

```
def acumula (fn, lst):
    Res = lst [0]
    for i in range (1, len (lst)):
        Res = fn (Res, lst [i])
    Return Res
```

## funções como valores de funções

→ ex: def derivada(x): def quadrado (y): >>> derivada (quadrado)(3)

. . . . . valor

valor = derivada do ponto $3^2$

## Recursão Funcional

→ O conceito de Repetição é Realizado exclusivamente através da Recursão;

exemplo: $x^n = \begin{cases} 1 & \text{se } n = 0 \\ x \times (x^{n-1}) & \text{se } n > 1 \end{cases}$

```
def potencia (x, n):
    if n == 0:
        Return 1
    else:
        RETURN x * potencia (x, n-1)
```

### Recursão Linear

→ Gera-se um encadeamento de operações suspensas à espera do valor de outras expressões:
1. Operação sucessivamente adiada (fase de expansão)
2. Execução das operações (fase de contração)

→ A um processo Recursivo que cresce linearmente com um valor dá-se o nome de processo Recursivo linear;

### Iteração Linear

```
→ def factorial-aux (prod-ac, prox):
    if prox == 0:
        Return prod-ac
    else:
        Return factorial-aux (prod_ac * prox, prox - 1)

def factorial (n):
    def factorial_aux (prod-ac, prox):
        . . . .

    Return factorial_aux (1, n)
```

→ Não se gera um encadeamento de funções suspensas / tem variáveis de estado e uma regra que especifica como atualizá-las

### Recursão com Processos e em funções

→ A evolução de processos pode ser classificada como uma evolução recursiva ou como uma evolução iterativa:

2. Um processo recursivo tem uma fase de expansão seguida de uma de contração. O computador mantém informação "escondida" que regista o ponto onde está o processo na cadeia de operações adiadas;
2. Um processo iterativo não cresce nem contrai

### Recursão em Árvore

→ Múltiplas faces de crescimento e contração → ineficiente

### Recursão de Cauda

→ Primeiro o cálculo é realizado, depois a chamada recursiva;

→ A chamada Recursiva é a última operação realizada pela função, não existindo operações adiadas;

```
def factorial(n, acc):
    if n == 0:
        Return acc
    else:
        Return factoria (n-1, n * acc)
```

fica

```
def factorial (n):
    def factorial_aux (n, acc):
        if n == 0:
            Retuen acc
        else:
            Return factorial-aux (n-1, n*acc)
    Return factorial-aux (n, 1)
```

### O Tipo Ficheiro

→ Localizar e utilizar: LER ou ESCREVER;

→ Open (abre o ficheiro)

→ 'R', 'W', 'A' aberto p/ leitura, aberto p/ escrita a partir do início, aberto p/ escrita a partir do fim

ff

→ encoding = <tipo>
↳ expressão que representa o tipo de codificação de caracteres utilizado no ficheiro

## Leitura de ficheiros

t = open ('teste.txt', 'R', encoding = 'UTF-16')
opcional

→ Operações:

1. t.Readline(): lê a linha do ficheiro que se encontra imediatamente a seguir ao indicador de leitura. se o indicador de leitura estiver no fim do ficheiro, esta função tem o valor ''.
2. t.Readlines(): Lê todos os caracteres do ficheiro que se encontram depois do indicador de leitura, tendo como valor uma lista em que cada um dos elementos é a cadeia de caracteres correspondente a cada uma das linhas que foi lida;
3. t.Read(): lê todos os caracteres do ficheiro, devolve cadeia de caracteres;
4. t.close(): fecha o ficheiro;

```
>>> f = open('teste.txt', 'R', encoding = 'UTF-16')
>>> l1 = f.readline()
>>> l1
'Este é um teste \n'
>>> l2 = f.Read()
>>> l2
'que mostra como o Python \nlê ficheiros'
>>> print(l2)
que mostra como o Python
lê ficheiros
```

```
>>> g = open('teste.txt', 'R', encoding = 'UTF-16')
>>> lines = g.Readlines()
['Este é um teste\n', 'que mostra como o Python \n', 'lê ficheiros de caracteres\n']
```

## Escrita em ficheiros

→ Operações:

1. t.write(char): escreve char em t. Devolve o nº de caracteres;
2. t.writelines(seq): seq = lista ou tuplos cujos elementos são char, não escreve o caractér de fim de linha; Não devolve valor;
3. t.close(): fecha o ficheiro;

→ Se um ficheiro é aberto para escrita, o seu conteúdo é apagado com a operação de abertura do ficheiro.

## O tipo Dicionário

→ dict
→ Tipo mutável, contém um conjunto de pares
- 1º elemento ↳ chave
- 2º elemento ↳ valor associado à chave

→ São representados dentro de chavetas { }
→ A chave tem de ser de um tipo imutável;

exemplos:

```
d = {'a':3, 'b':2, 'c':4}        d[c] = 4
>>> ex_dic['a'] = ex_dic['a'] + 1
>>> ex_dic['a']
4
```

| Operação | Valor |
|---|---|
| c in d | True se a chave c pertence ao dicionário |
| c not in d | True se a chave c não pertence a d |
| len(d) | Nº de elementos de d |

```
n = {'a':1, 'b':2, 'c':3}
>>> for i in n:
        print(i, n[i])
a 1
b 2
c 3
```

## Contagem de letras num texto

```
lus = open('Lusiadas.txt', 'r', encoding = 'UTF-16')
cont = {}
linha = lus.readline
while linha != '':
    for c in linha:
        if c not in cont:
            cont[c] = 1
        else:
            cont[c] = cont[c] + 1
    linha = lus.Readline()
lus.close()
```

```
>>> ex_cadeia = 'Exemplo DE CONVERSÃO'
>>> ex_cadeia.lower()
'exemplo de conversão'
```

## Dicionários de Dicionários

→ {'FP': {'2010/11': 12, '2011/12': 15}, 'AL': {'2010/11': 10}, 'TC': {'2010/11": 12}, 'SD': {'2010/11': 'REP', '2011/12': 13}}

## A Abstração em Programação

→ A abstração de dados é uma metodologia que permite separar o modo como uma estrutura de informação é utilizada dos pormenores relacionados com o modo como essa estrutura de informação é construída a partir de outras estruturas de informação

## Tipos Abstratos de Informação

→ Separação das partes dos programas que lidam com o modo como as entidades do tipo são utilizadas das partes que lidam com o modo como as entidades são representadas.

1. Identificação das operações básicas
2. Axiomatização das operações básicas
3. Escolha de uma representação
4. Concretização das op. básicas

## Identificação das Operações Básicas

→ Construtores: constroem novos elementos do tipo;

→ Selectores: Acedem aos constituintes dos elementos do tipo;

→ Modificadores: Alteram destrutivamente os elementos do tipo, ex: del;

→ Transformadores: Transformam os elementos de um tipo noutro tipo;

→ Reconhecedores: Identificam elementos do tipo;

→ Testes: comparações entre os elementos do tipo;

## Objetos

→ Entidade computacional que apresenta informação interna e um conjunto de operações que definem o seu comportamento;

1. A palavra class indica que se trata da definição de um objeto; pode apenas ser definido um nome; pode ser fornecido um nome seguido de outro nome entre () ; representa o nome do objeto
2. Definição de um método que está associado ao objeto; existe sempre um CUJO
3. Os parâmetros formais de um método contém sempre, como primeiro elemento, um parâmetro com o nome self.

```
class compl:  → nºs complexos
    def __init__(self, Real, imag):
        if isinstance(Real, (int, float)) and isinstance(imag, (int, float)):
            self.R = Real
            self.i = imag
        else:
            Raise ValueError ('complexo: argumento inválido')

    def p_Real(self):
        Return self.R
```

```
def p_imag(self):
    Return self.i

def compl_zero(self):
    Return self.R==0 and self.i==0

def imag_puro(self):
    Return self.R==0

def compl_iguais(self, outro):
    Return self.R == outro.p_Real() and self.i == outro.p_imag()

def escreve(self):
    if self.i >=0:
        print(self.R, '+', self.i, 'i')
    else:
        print(self.R, '-', abs(self.i), 'i')
```

→ c1.p_Real é o nome do método que devolve a parte real de c1

```
>>> c1 = compl(3,2)
>>> c1.p_Real()
3
>>> c2.p_imag()
2
>>> c1.escreve()
3+2i
>>> c2 = compl(2,-2)
>>> c2.escreve()
2-2i
```

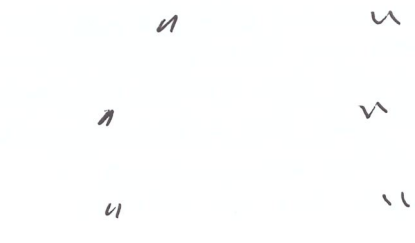
```
>>> c1 = compl(3,5)
>>> type(c1)
<class '__main__.compl'>
>>> c1 = compl(9,6)
>>> isinstance(c1, compl)
True
```

→ Uma classe corresponde a uma infinidade de objetos com as mesmas variáveis e com o mesmo comportamento.

## ° O Tipo Conta Bancária

→

```
class conta:
    def __init__(self, quantia):
        self.saldo = quantia
    def consulta(self):
        Return self.saldo
    def deposito(self, quantia):
        self.saldo = self.saldo + quantia
        Return self.saldo
    def levantamento(self, quantia):
        if self.saldo - quantia >= 0:
            self.saldo = self.saldo - quantia
            Return self.saldo
        else:
            print('Saldo insuficiente')
```




```
def levantamento(self, quantia):
    self.saldo = self.saldo - quantia
    Return self.saldo
```

## ° Classe, Subclasse e Heranças

→ Uma classe é uma subclasse de outra classe, se a primeira corresponder a uma especialização da segunda; o comportamento da subclasse corresponde ao comportamento da superclasse, exceto no caso em que o comportamento específico está indicado para a subclasse; A subclasse herda o comportamento da superclasse;

```
class conta-ordenado (conta-gen):

    def __init__(self, quantia, ordenado):
        if quantia >= ordenado:
            self.saldo = quantia
            self.ordenado = ordenado
        else:
            print('O saldo deve ser maior que o ordenado')

    def levantamento(self, quantia):
        if quantia <= self.saldo + self.ordenado:
            self.saldo = self.saldo - quantia
            Return self.saldo
        else:
            print('Saldo insuficiente')
```

ff

## Polimorfismo

→ Uma operação é polimórfica quando é possível definir funções diferentes que usam a mesma operação para lidar com tipos de dados diferentes;

```
>>> a=5
>>> a.__add__(2)
7
```

```
__sub__  -      __eq__  ==
__mul__  *      __repr__ → char
__truediv__ /
```

```
def __add__(self, outro):
    R = self.p_real() + outro.p_real()
    i = self.p_imag() + outro.p_imag()
    return compl(R,i)
```

```
>>> c1 = compl(2,4)
>>> c2 = comp(5,10)
>>> c3 = c1 + c2
>>> c3.escrere()
7 + 14i
```

O + é o add

```
class compl:
    def __init__(self, real, imag):
        if isinstance(real, (int, float)) and isinstance(imag, (int, float)):
            self.R = real
            self.i = imag
        else:
            raise ValueError('complexo: arg errado')
    def p_real(self):
        return self.R
    def p_imag(self):
        return self.i
    def compl_zero(self):
        return self.R == 0 and self.i == 0
    def __eq__(self, outro):
        return self.R == outro.preal() and self.i == outro.p_imag()
    def __add__(self, outro):
        R = self.p_real() + outro.p_real()
        i = self.p_imag() + outro.p_imag()
        return compl(R, i)
```

## Pilhas

→ Estruturas de informação constituídas por uma sequência de elementos; são retirados pela ordem inversa pela qual foram colocados, põe-se no topo, tira-se do topo; Não se pode aceder ou inspecionar nenhum elemento exceto o que está no topo

## Operações Básicas para Pilhas

IMUTÁVEIS / MUTÁVEIS

→ Como Imutáveis:

1. Construtores: constroem pilhas → cria pilha (nova-pilha) e insere elemento (empurra) (no topo)
2. Seletores: selecionam partes → indica o topo (topo) e remove (tira)
3. Reconhecedores: identificam tipos de pilhas (pilha) → tipo e vazia (pilha-vazia)
4. Testes: iguais (pilhas-iguais)

→ Como Mutáveis:

1. Construtores: nova-pilha
2. Modificadores: empurra (+ el no topo) e tira (remove o topo)
3. Seletores: topo (inaica o el no topo)
4. Reconhecedores: = imutáveis
5. Testes: = imutáveis

→ A pilha com elementos 3 5 7 (em que 3 está no topo) é representada:

3
5
7
= = =

## Axiomatização

→ Devem verificar-se as seguintes relações:

pilha(nova-pilha()) = verdadeiro

pilha(empurra(p,e)) = verdadeiro

pilha(tira(p)) = { verdadeiro se p ñ vazia; ⊥ caso contrário }

pilha-vazia(nova-pilha()) = verd

pilha-vazia(empurra(p,e)) = falso

topo(empurra(p,e)) = e

tira(empurra(p,e)) = p

pilhas-iguais(p1,p2) = { pilha-vazia(p1) se pilha-vazia(
pilha-vazia(p1) se pilha-vazia(p
topo(p1) = topo(p2) se
tira(p1) = tira(p2)
falso caso contrário

## Representação de Pilhas

→ Recorrendo a listas:

1. Pilha vazia é a lista vazia;
2. O primeiro elemento corresponde ao topo;

## Realização de Operações Básicas

```
>>> teste = [1,2]
>>> teste[2] >0
IndexError: list index out of range
>>> try:
        teste[2] >0
    except IndexError:
        print ('Enganou-se no indice')
    Enganou-se no indice

def nova-pilha():
    Return []
def empurra (pilha, elementos):
    Return [elemento] + pilha
def topo (pilha):
    try:
        RETURN pilha[0]
    except IndexError:
        Raise ValueError ('A pilha não tem elementos')
def tira (pilha):
    try:
        Return pilha[1:]
    except IndexError:
        Raise ValueError ('A pilha nte')
def pilha(x):
    if x == []:
        Return True
    elif isinstance (x, list):
        Return True
    else:
        Return False
def pilha-vazia (pilha):
    Return pilha == []
def pilhas-iguais (p1, p2):
    Return p1 == p2
def mostra-pilha (pilha):
    if pilha != []:
        for e in pilha:
            print (e)
    Print ('===')
```

---

```
class pilha:
    def __init__ (self):
        self.p = []
    def empurra (self, elemento):
        self.p = [elemento] + self.p
        Return self.
    def tira (self):
        try:
            del (self.p[0])
            Return self
        except IndexError:
            print ('APUTE')
```

```
(self):
    return self.p == []

-- Repr -- (self):
    if self.p != []:
        Rep = ''
        for e in self.p:
            Rep = Rep + ' ' + str(e) + '\n'
        Rep = Rep + '===='
        Return Rep
    else:
      Return '===='
```

- Filas
  - → Sequência de elementos retirados pela ordem em que foram colocados; Apenas o 1º elemento pode ser acedido ou removido;
- Operações Básicas para filas
  - → 1. Construtores: nova-fila
    2. Seletores: início e fim
    3. Modificadores: coloca (novos elem.) e retira
    4. Transformadores: produz uma lista com os el da fila
    5. Reconhecedores;
    6. Testes: filas-iguais;
- A Classe Filas
  - → class fila:

```
def __init__(self):
    self.f = []

def inicio(self):
    try:
        Return self.f[0]
    except IndexError:
        print('inicio: a fila ñ t el')

def comprimento(self):
    Return len(self.f)

def coloca(self, elemento):
    self.f = self.f + [elemento]
    Return self.f

def retira(self):
    try:
        del(self.f[0])
        Return self
    except IndexError:
        print('...')

def fila-p-lista(self):
    lst = []
    for i in range(len(self.f)):
        lst = lst + [self.f[i]]
    Return lst

def fila-vazia(self):
    Return self.f == []

def filas-iguais(self, outra):
    outra-lista = outra.fila-para-lista()
    if len(self.f) != len(outra-lista):
        Return false
    else:
        for i in range(len(self.f)):
            if self.f[i] != outra-lista[i]:
                return false
        Return True
```

```
def __repr__(self):
    f = '<'
    if self.f != []:
        for i in range(len(self.f)):
            f = f + self.f[i].__repr__() + ','
    f = f + '>'
    return f
```

- Funcionais sobre Listas
  - Filtra, Transforma, Acumula
  - filter, map, Reduce

```
(lambda x,y: x+y)(1,2)
>>> 3
```

```
lambda x: x * fatorial(x-1) if x>1 else 1
x=4
>>> 24
```

map ( lambda x : x**3 , range(5,10)) → lista com $5^3, \ldots, 9^3$

Reduce ( lambda x,y: x+ , [1,2,3] → 6

filter ( lambda x: x > 50 , [1,44,60] ) → [60]

Maria Ribeiro
10/01/2019